DQMH Project Ejercicio 1

Antidoc v3.3.2, Enrique Noe Arias

Table of Contents

1. Project description . . . 1
2. Stand-alone VIs . . . 2
   2.1. Testers . . . 2
3. DQMH® modules . . . 3
   3.1. Preamble . . . 3
   3.2. Modules overview . . . 5
   3.3. My Singleton Module.lvlib . . . 6
   3.4. My Cloneable Module.lvlib . . . 11
4. Libraries . . . 18
   4.1. Application.lvlib . . . 18
5. Custom errors . . . 20
6. Bookmarks . . . 21
7. Legal Information . . . 25
   7.1. Document creation . . . 25
   7.2. Product used in the project . . . 27

Section 1. Project description

No description found (add content in project description)

Section 2. Stand-alone VIs

This project has 2 VIs (or typedefs) that do not belong to any library.

2.1. Testers

Table 1. VIs and Typedef

Name	Connector pane	Description	R.	I.
Test My Singleton Module API.vi		My Singleton Module API Tester.		
Test My Cloneable Module API.vi		My Module API Tester.		

Reentrancy: → Preallocated reentrancy | → Shared reentrancy

Inlining: → Inlined

Section 3. DQMH® modules

This section describes DQMH® module responsibilities and relationships.

3.1. Preamble

A DQMH module is the main component of an architecture based on DQMH® framework. A DQMH module is used to implement a section of the application that has one responsibility.

DQMH® framework defines two different type of DQMH module.

Singleton:

A Singleton DQMH module can have only one instance running at any given time.

Cloneable:

A Cloneable DQMH module can have one or multiple instances running in parallel.

DQMH® framework defines two different ways to carry data throughout the application and with both other DQMH modules and non-DQMH based code.

Request events:

A request is a code that fires an event requesting the DQMH module to do something. Multiple locations in the code can send events to the DQMH module.

Request events are many-to-one.

Requests are usually named using imperative tense.

Broadcast events:

A broadcast is a code that fires an event broadcasting that the DQMH module did something. Multiple Event Structures can register to handle the Broadcast Events.

Broadcast Events are one-to-many.

Broadcasts are usually named using past tense or passive voice.

Default events:

By default all the DQMH ® Modules are shipped with some default events (Requests and Broadcasts). Here is the list of this events:

- Start Module
- Stop Module
- Show Panel
- Hide Panel
- Show Diagram
- Status Updated
- Error Reported
- Module Did Stop
- Update Module Execution Status
- Module Did Init
- Get Module Execution Status

If you want to learn more about default DQMH ® Events see the documentaiton.

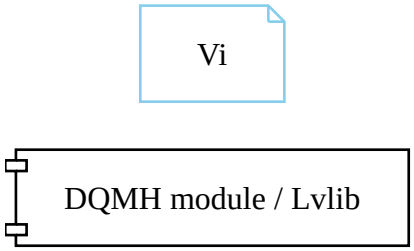
NOTE	Refer to the DQMH® framework official documentation to find more details on how the framework works

The following section gives you details on the project architecture relying on this framework. It gives you an overview of the modules' interaction and detailed information on each module.

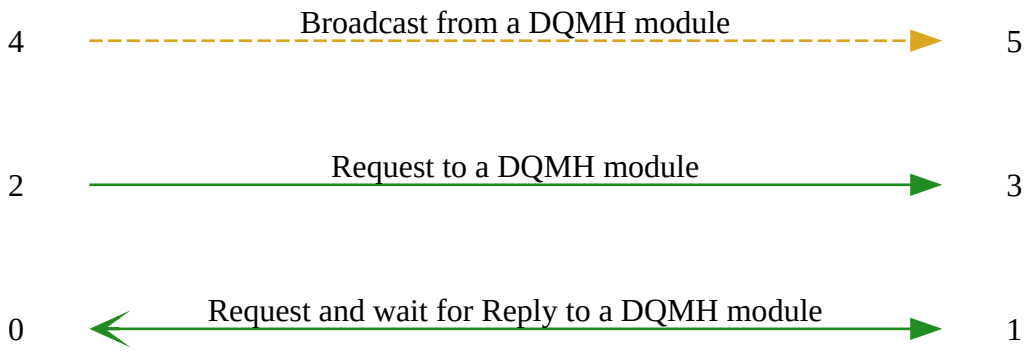
NOTE	If your module has a helper loop, it will be listed along with the events it processes. Helper loops can be categorized as either DQMH or Custom. To ensure your helper loop adheres to DQMH guidelines see the documentaiton.

Graphs used in this section have the following legend:

Components:

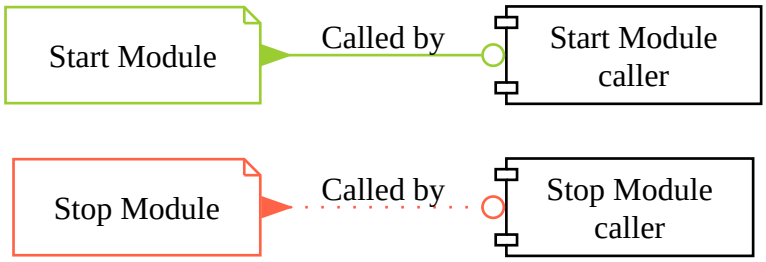


Events:



NOTE	One arrow can represent one or more events between two components
NOTE	Request and Request and wait for Reply are represented by only one arrow. If there is no Request and wait for Reply, Request representation is used. Otherwise Request and wait for Reply is used

Start and Stop module callers:



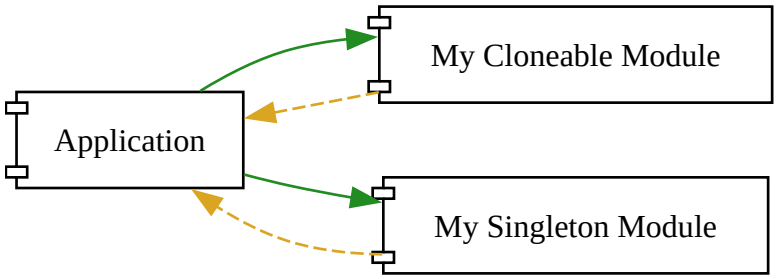
3.2. Modules overview

This project contains 1 singleton module and 1 cloneable module.

Table 2. Modules list

Singleton	Cloneable
My Singleton Module.lvlib	My Cloneable Module.lvlib

This graph represents the links between all DQMH modules.



3.3. My Singleton Module.lvlib

Type: Singleton

Responsibility: No description found (add content in DQMH module lvlib description)

3.3.1. Module relationship

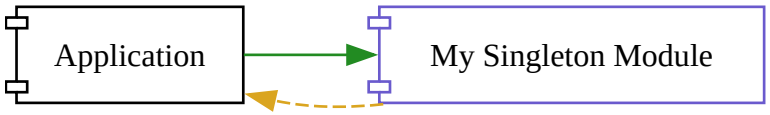


Table 3. Requests callers

Request Name	Callers
Do Something Else and Wait for Reply	
Do Something Else	Application.lvlib:Main.vi
Do Something	Application.lvlib:Main.vi
Get Module Execution Status	
Hide Panel	Application.lvlib:Main.vi
Show Diagram	
Show Panel	Application.lvlib:Main.vi

Table 4. Broadcasts Listeners

Broadcast Name	Listeners
Did Something	
Error Reported	Application.lvlib:Main.vi
Module Did Init	Application.lvlib:Main.vi
Module Did Stop	Application.lvlib:Main.vi
Status Updated	Application.lvlib:Main.vi
Update Module Execution Status	Application.lvlib:Main.vi

Table 5. Used requests

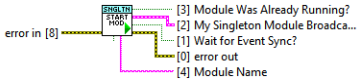
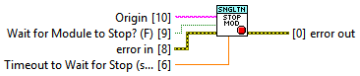
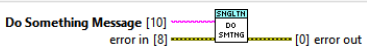
Module	Requests
—	—

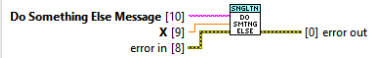
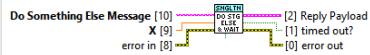
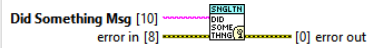
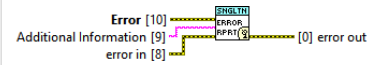
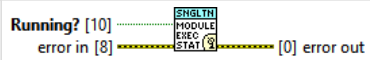
Table 6. Registered broadcast

Module	Broadcasts
—	—

3.3.2. Event list

Table 7. Events

Name	Type	Connector pane	Description	S.	R.	I.
Start Module			Launches the module Main VI. After calling this VI, you can optionally register for broadcast events from the module by wiring the broadcast events output of this VI to a **Register For Events** function. After the optional Register For Events function call, you should always call the **Synchronize Module Events.vi** for this module with the 'Wait for Event Sync?' output of this VI to the corresponding input of the Synchronize Module Events.vi. To see an example of the proper wiring pattern, see the "Start Module: Value Change" event frame in the API Tester VI for this module. The **Validate DQMH Module** tool added a 'Module Name' output to this VI.			
Stop Module			Send the Stop request to the Module's Main.vi. If **Wait for Module to Stop?** is TRUE, this VI will wait until the module main VI stops, and will timeout at the **Timeout to Wait for Stop** value. This value defaults to "-1", which means the VI will not timeout, and will always wait until the module main VI stops before completing execution. Note: The **Timeout to Wait for Stop** value is ignored if 'Wait for Module to Stop?' is set to FALSE.			
Do Something			Send the Do something request to the Module's Main.vi.			

Name	Type	Connector pane	Description	S.	R.	I.
Do Something Else			Send the Do Something Else request to the Module's Main.vi.			
Do Something Else and Wait for Reply			Send the Do Something Else request to the Module's Main.vi.			
Show Panel			Send the Show Panel request to the Module's Main.vi.			
Hide Panel			Send the Hide Panel request to the Module's Main.vi.			
Get Module Execution Status			Fire the Get Module Execution Status request.			
Show Diagram			This VI tells the Module to show its block diagram to facilitate troubleshooting (add probes, breakpoints, highlight execution, etc).			
Module Did Init			Send the Module Did Init event to any VI registered to listen to this module's broadcast events.			
Did Something			Send the Did Something event to any VI registered to listen to this module's broadcast events.			
Status Updated			Send the Status Updated event to any VI registered to listen to events from the owning module.			
Error Reported			Send the Error Reported event to any VI registered to listen to events from the owning module.			
Module Did Stop			Send the Module Did Stop event to any VI registered to listen to this module's broadcast events.			
Update Module Execution Status			Broadcast event to specify whether or not the module is running.			

Type: → Request | → Request and Wait for Reply | → Broadcast

Scope: → Protected | → Community

Reentrancy: → Preallocated reentrancy | → Shared reentrancy

Inlining: → Inlined

3.3.3. Module Start/Stop calls

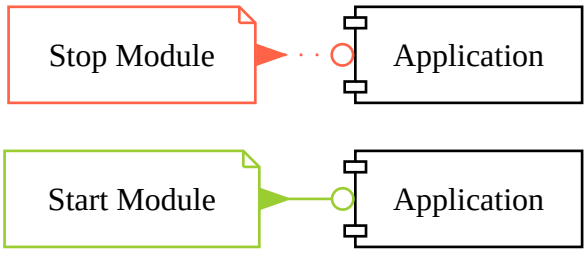


Table 8. Start and Stop module callers

Function	Callers
Start Module	Application.lvlib:Main.vi
Stop Module	Application.lvlib:Main.vi

3.3.4. Module Helper Loops

NOTE No Helper Loops Found

3.3.5. Module custom errors

TIP Custom errors are added to the module via vi named `*--error.vi`.

Module My Singleton Module.lvlib use the following custom errors:

Table 9. Custom errors

Name	Code	Description
Module Not Running	403681	%s Module is not running.
Module Not Stopped	403682	The Stop Module VI for the %s module timed out while waiting for the module main VI to stop. The module main VI may still be running.
Module Not Synced	403683	%s Module was unable to synchronize events.
Request and Wait for Reply Timeout	403686	

3.3.6. Module Constant VIs

Table 10. Constant VIs Found

VI Name	Data Type	Value
Module Name--constant	String	My Singleton Module
Module Timeout--constant	I32	5000

3.3.7. Bookmarks

#DQMH_CodeNeeded

Text	VI
#DQMH_CodeNeeded - If you add controls to the connector pane of Main.vi in order to pass in values when starting the module, drag the Main.vi for this DQMH module onto this constant to update the connector pane of the Start on Asynchronous Call function.	Start Module.vi
#DQMH_CodeNeeded - Initialize your state data here.	Main.vi
#DQMH_CodeNeeded - Initialize the appearance of the front panel.	Main.vi
#DQMH_CodeNeeded - Add any required cleanup code here.	Main.vi

#DQMH_CodeRecommended

Text	VI
#DQMH_CodeRecommended - If you rename your module, update the constant value above.	Module Name--constant.vi
#DQMH_CodeRecommended - If the module is running in standalone mode (External Launch = FALSE), this code will pop up an error dialog and call the 'Exit' message if an error occurs. Add more fine-grained error management here.	Main.vi

#DQMH_HowTo

Text	VI
#DQMH_HowTo This is the top-level VI of a DQMH Singleton module. See the following video for a walkthrough of this VI's functionality: https://bit.ly/DQMHMainWalkthrough	Main.vi
#DQMH_HowTo The 'Simulate MHL Error' button illustrates how the MHL handles errors. This event is instructional in nature and can be removed.	Main.vi
#DQMH_HowTo The 'Simulate MHL Error' button illustrates how the MHL handles errors. This event is instructional in nature and can be removed.	Main.vi
#DQMH_HowTo The 'Simulate EHL Error' button illustrates how the EHL handles errors. This event is instructional in nature and can be removed.	Main.vi

Text	VI
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event Remove DQMH Event utility.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi

3.4. My Cloneable Module.lvlib

Type: Cloneable

Responsibility: No description found (add content in DQMH module lvlib description)

3.4.1. Module relationship

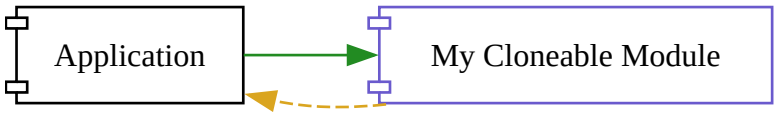


Table 11. Requests callers

Request Name	Callers
Do Something Else and Wait for Reply	
Do Something Else	Application.lvlib:Main.vi
Do Something	Application.lvlib:Main.vi

Request Name	Callers
Hide Panel	Application.lvlib:Main.vi
Show Diagram	
Show Panel	Application.lvlib:Main.vi

Table 12. Broadcasts Listeners

Broadcast Name	Listeners
Did Something	
Error Reported	Application.lvlib:Main.vi
Module Did Init	Application.lvlib:Main.vi
Module Did Stop	Application.lvlib:Main.vi
Status Updated	Application.lvlib:Main.vi
Update Module Execution Status	

Table 13. Used requests

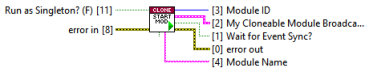
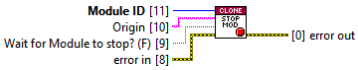
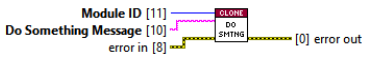
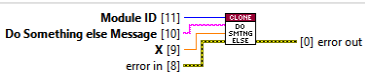
Module	Requests
—	—

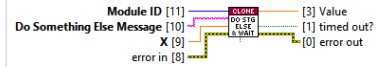
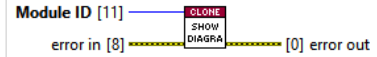
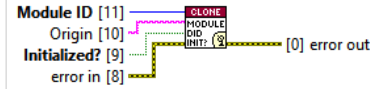
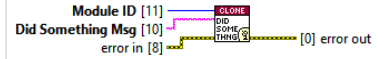
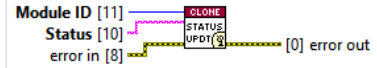
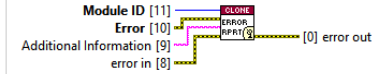
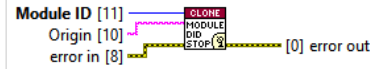
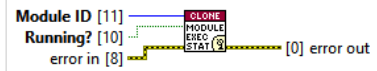
Table 14. Registered broadcast

Module	Broadcasts
—	—

3.4.2. Event list

Table 15. Events

Name	Type	Connector pane	Description	S.	R.	I.
Start Module			Launches an instance of the module Main VI. After calling this VI, you can optionally register for broadcast events from the module by wiring the broadcast events output of this VI to a **Register For Events** function. After the optional Register For Events function call, you should always call the **Synchronize Module Events.vi** for this module with the 'Wait for Event Sync?' output of this VI to the corresponding input of the Synchronize Module Events.vi. To see an example of the proper wiring pattern, see the "Run New Module: Value Change" event frame in the API Tester VI for this module. The **Validate DQMH Module** tool added a 'Module Name' output to this VI.			
Stop Module			Send the Stop request to the Module's Main.vi. If **Wait for Module to stop?** is TRUE, then this VI will not complete execution until the Module Main VI has stopped running. **Note:** If the cloneable module is running as singleton, then the 'Wait for Module to stop?' input is ignored… this VI will **always** wait until a cloneable Main VI running as singleton has stopped running.			
Do Something			Send the Do something request to the Module's Main.vi.			
Do Something Else			Send the Do Something Else request to the Module's Main.vi.			

Name	Type	Connector pane	Description	S.	R.	I.
Do Something Else and Wait for Reply			Send the Do Something Else request to the Module's Main.vi.			
Show Panel			Send the Show Panel request to the Module's Main.vi.			
Hide Panel			Send the Hide Panel request to the Module's Main.vi.			
Show Diagram			This VI tells the Module to show its block diagram to facilitate troubleshooting (add probes, breakpoints, highlight execution, etc).			
Module Did Init			Send the Module Did Init event to any VI registered to listen to this module's broadcast events.			
Did Something			Send the Did Something event to any VI registered to listen to this module's broadcast events.			
Status Updated			Send the Status Updated event to any VI registered to listen to events from the owning module.			
Error Reported			Send the Error Reported event to any VI registered to listen to events from the owning module.			
Module Did Stop			Send the Module Did Stop event to any VI registered to listen to this module's broadcast events.			
Update Module Execution Status			Fire the Get Module Execution Status request.			

Type: → Request | → Request and Wait for Reply | → Broadcast

Scope: → Protected | → Community

Reentrancy: → Preallocated reentrancy | → Shared reentrancy

Inlining: → Inlined

3.4.3. Module Start/Stop calls

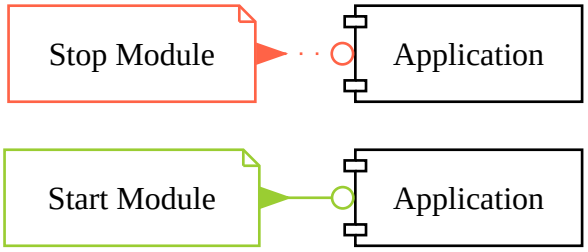


Table 16. Start and Stop module callers

Function	Callers
Start Module	Application.lvlib:Main.vi
Stop Module	Application.lvlib:Main.vi

3.4.4. Module Helper Loops

NOTE No Helper Loops Found

3.4.5. Module custom errors

TIP Custom errors are added to the module via vi named `*--error.vi`.

Module My Cloneable Module.lvlib use the following custom errors:

Table 17. Custom errors

Name	Code	Description
Module Running as Singleton	403680	The "%s" module is currently running as singleton, but the Start Module VI was called with 'Run as Singleton' specified as FALSE.
Module Not Stopped	403682	The Stop Module VI for the %s module timed out while waiting for the module main VI to stop. The module main VI may still be running.
Module Not Synced	403683	%s Module was unable to synchronize events.
Module Not Running	403684	Not a single instance of "%s" Module running.
Module Running as Cloneable	403685	The "%s" module is currently running as cloneable, but the Start Module VI was called with 'Run as Singleton' specified as TRUE.
Request and Wait for Reply Timeout	403686	

Name	Code	Description
Master Reference Not Closed	403687	The "%s" module cannot be run as singleton because the Master Reference is still open from a prior run as cloneable. If you plan on running this module as both singleton and cloneable, consider changing your Main VI to wire a value of TRUE to the 'Close Master Reference' input of Init Module.vi.

3.4.6. Module Constant VIs

Table 18. Constant VIs Found

VI Name	Data Type	Value
Module Name--constant		
Module Timeout--constant	I32	5000

3.4.7. Bookmarks

#DQMH_CodeNeeded

Text	VI
#DQMH_CodeNeeded - When adding/changing parameters on the Main VI connector pane: 1. Right-click this constant and select Open Type Def. 2. Drag the Main VI for this DQMH module onto the Main VI Type control inside the type definition. 3. Save and close the type definition.	Start Module.vi
#DQMH_CodeNeeded - Initialize your state data here.	Main.vi
#DQMH_CodeNeeded - Initialize the appearance of the front panel.	Main.vi
#DQMH_CodeNeeded - Add any required cleanup code here.	Main.vi

#DQMH_CodeRecommended

Text	VI
#DQMH_CodeRecommended - If you rename your module, update the constant value above.	Module Name--constant.vi
#DQMH_CodeRecommended - If the module is running in standalone mode (External Launch = FALSE), this code will pop up an error dialog and call the 'Exit' message if an error occurs. Add more fine-grained error management here.	Main.vi

#DQMH_HowTo

Text	VI
#DQMH_HowTo This is the top-level VI of a DQMH Cloneable module. See the following video for a discussion of cloneable-specific functionality: https://bit.ly/DQMHCloneableMainWalkthrough	Main.vi
#DQMH_HowTo The 'Simulate MHL Error' button illustrates how the MHL handles errors. This event is instructional in nature and can be removed.	Main.vi
#DQMH_HowTo The 'Simulate MHL Error' button illustrates how the MHL handles errors. This event is instructional in nature and can be removed.	Main.vi
#DQMH_HowTo The 'Simulate EHL Error' button illustrates how the EHL handles errors. This event is instructional in nature and can be removed.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi

Section 4. Libraries

This section describes the libraries contained in the project.

4.1. Application.lvlib

Responsibility: No description found (add content in lvlib description)

Version: 1.0.0.0

4.1.1. Functions

Table 19. Functions (non private scope only)

Name	Connector pane	Description	S.	R.	I.
Main		This VI is the main application VI in the DQMH project template. This application VI is implemented as a simple state machine, which communicates with two different DQMH modules.			

Scope: → Protected | → Community

Reentrancy: → Preallocated reentrancy | → Shared reentrancy

Inlining: → Inlined

4.1.2. Library Constant VIs

Table 20. Constant VIs Found

VI Name	Data Type	Value
Application Name--constant	String	Application

4.1.3. Bookmarks

#DQMH_HowTo

Text	VI
#DQMH_HowTo This VI is based on the Simple State Machine project template. It illustrates how to use non-DQMH code to call DQMH modules. In this simple state machine VI (which is the top-level application for the DQMH project template), there are several calls to the Public API VIs of the Singleton and Cloneable DQMH modules that are also part of the DQMH project template. For more illustrations of how to call DQMH modules from non-DQMH applications, browse the DQMH examples located at [LabVIEW 20xx]\examples\DQMH Consortium. See the following video for a walkthrough of this application functionality: https://bit.ly/DQMHProjectTemplateApplicationWalkthrough Documentation available via Help > DQMH Consortium > DQMH Help… menu.	Main.vi

Section 5. Custom errors

TIP Custom errors are added via vi named `*--error.vi`.

Table 21. Custom errors

Name	Code	Description	Owned by
Module Running as Singleton	403680	The "%s" module is currently running as singleton, but the Start Module VI was called with 'Run as Singleton' specified as FALSE.	My Module.lvlib Cloneable
Module Not Running	403681	%s Module is not running.	My Module.lvlib Singleton
Module Not Stopped	403682	The Stop Module VI for the %s module timed out while waiting for the module main VI to stop. The module main VI may still be running.	My Module.lvlib Singleton My Module.lvlib Cloneable
Module Not Synced	403683	%s Module was unable to synchronize events.	My Module.lvlib Singleton My Module.lvlib Cloneable
Module Not Running	403684	Not a single instance of "%s" Module running.	My Module.lvlib Cloneable
Module Running as Cloneable	403685	The "%s" module is currently running as cloneable, but the Start Module VI was called with 'Run as Singleton' specified as TRUE.	My Module.lvlib Cloneable
Request and Wait for Reply Timeout	403686		My Module.lvlib Singleton My Module.lvlib Cloneable
Master Reference Not Closed	403687	The "%s" module cannot be run as singleton because the Master Reference is still open from a prior run as cloneable. If you plan on running this module as both singleton and cloneable, consider changing your Main VI to wire a value of TRUE to the 'Close Master Reference' input of Init Module.vi.	My Module.lvlib Cloneable

Section 6. Bookmarks

6.1. #DQMH_CodeNeeded

Text	VI	Owned by
#DQMH_CodeNeeded - When adding/changing parameters on the Main VI connector pane: 1. Right-click this constant and select Open Type Def. 2. Drag the Main VI for this DQMH module onto the Main VI Type control inside the type definition. 3. Save and close the type definition.	Start Module.vi	My Cloneable Module.lvlib
#DQMH_CodeNeeded - If you add controls to the connector pane of Main.vi in order to pass in values when starting the module, drag the Main.vi for this DQMH module onto this constant to update the connector pane of the Start on Asynchronous Call function.	Start Module.vi	My Singleton Module.lvlib
#DQMH_CodeNeeded - Initialize your state data here.	Main.vi	My Singleton Module.lvlib
#DQMH_CodeNeeded - Initialize your state data here.	Main.vi	My Cloneable Module.lvlib
#DQMH_CodeNeeded - Initialize the appearance of the front panel.	Main.vi	My Singleton Module.lvlib
#DQMH_CodeNeeded - Initialize the appearance of the front panel.	Main.vi	My Cloneable Module.lvlib
#DQMH_CodeNeeded - Add any required cleanup code here.	Main.vi	My Singleton Module.lvlib
#DQMH_CodeNeeded - Add any required cleanup code here.	Main.vi	My Cloneable Module.lvlib

6.2. #DQMH_CodeRecommended

Text	VI	Owned by
#DQMH_CodeRecommended - If you rename your module, update the constant value above.	Module Name--constant.vi	My Singleton Module.lvlib
#DQMH_CodeRecommended - If you rename your module, update the constant value above.	Module Name--constant.vi	My Cloneable Module.lvlib
#DQMH_CodeRecommended - If the module is running in standalone mode (External Launch = FALSE), this code will pop up an error dialog and call the 'Exit' message if an error occurs. Add more fine-grained error management here.	Main.vi	My Singleton Module.lvlib

Text	VI	Owned by
#DQMH_CodeRecommended - If the module is running in standalone mode (External Launch = FALSE), this code will pop up an error dialog and call the 'Exit' message if an error occurs. Add more fine-grained error management here.	Main.vi	My Cloneable Module.lvlib
#DQMH_CodeRecommended - If you rename the Application, update the constant value above.	Application Name--constant.vi	Application.lvlib

6.3. #DQMH_HowTo

Text	VI	Owned by
#DQMH_HowTo This is the top-level VI of a DQMH Singleton module. See the following video for a walkthrough of this VI's functionality: https://bit.ly/DQMHMainWalkthrough	Main.vi	My Singleton Module.lvlib
#DQMH_HowTo This is the top-level VI of a DQMH Cloneable module. See the following video for a discussion of cloneable-specific functionality: https://bit.ly/DQMHCloneableMainWalkthrough	Main.vi	My Cloneable Module.lvlib
#DQMH_HowTo This VI is based on the Simple State Machine project template. It illustrates how to use non-DQMH code to call DQMH modules. In this simple state machine VI (which is the top-level application for the DQMH project template), there are several calls to the Public API VIs of the Singleton and Cloneable DQMH modules that are also part of the DQMH project template. For more illustrations of how to call DQMH modules from non-DQMH applications, browse the DQMH examples located at [LabVIEW 20xx]\examples\DQMH Consortium. See the following video for a walkthrough of this application functionality: https://bit.ly/DQMHProjectTemplateApplicationWalkthrough Documentation available via Help > DQMH Consortium > DQMH Help... menu.	Main.vi	Application.lvlib
#DQMH_HowTo The 'Simulate MHL Error' button illustrates how the MHL handles errors. This event is instructional in nature and can be removed.	Main.vi	My Singleton Module.lvlib
#DQMH_HowTo The 'Simulate MHL Error' button illustrates how the MHL handles errors. This event is instructional in nature and can be removed.	Main.vi	My Singleton Module.lvlib
#DQMH_HowTo The 'Simulate MHL Error' button illustrates how the MHL handles errors. This event is instructional in nature and can be removed.	Main.vi	My Cloneable Module.lvlib

Text	VI	Owned by
#DQMH_HowTo The 'Simulate MHL Error' button illustrates how the MHL handles errors. This event is instructional in nature and can be removed.	Main.vi	My Cloneable Module.lvlib
#DQMH_HowTo The 'Simulate EHL Error' button illustrates how the EHL handles errors. This event is instructional in nature and can be removed.	Main.vi	My Singleton Module.lvlib
#DQMH_HowTo The 'Simulate EHL Error' button illustrates how the EHL handles errors. This event is instructional in nature and can be removed.	Main.vi	My Cloneable Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed.	Main.vi	My Singleton Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed.	Main.vi	My Cloneable Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed.	Main.vi	My Cloneable Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event Remove DQMH Event utility.	Main.vi	My Singleton Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi	My Singleton Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi	My Singleton Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi	My Singleton Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi	My Singleton Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi	My Singleton Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi	My Singleton Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi	My Cloneable Module.lvlib

Text	VI	Owned by
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi	My Cloneable Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi	My Cloneable Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi	My Cloneable Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi	My Cloneable Module.lvlib
#DQMH_HowTo The 'Do Something' messages are instructional in nature and can be removed with the Tools > DQMH Consortium > DQMH > Event > Remove DQMH Event utility.	Main.vi	My Cloneable Module.lvlib

Section 7. Legal Information

7.1. Document creation

This document has been generated using the following tools.

7.1.1. Antidoc

Project website: Antidoc

Maintainer website: Wovalab

BSD 3-Clause License

Copyright © 2019-2025, Wovalab, All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of the copyright holder nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

7.1.2. Asciidoc for LabVIEW™

Project website: Asciidoc toolkit

Maintainer website: Wovalab

BSD 3-Clause License

Copyright © 2019-2025, Wovalab, All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of the copyright holder nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

7.1.3. Graph Builder

Project website: Graph Builder

BSD 3-Clause License

Copyright © 2020, Cyril GAMBINI All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of the copyright holder nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES

(INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

7.2. Product used in the project

The documented project has been developed with the following products.

7.2.1. DQMH®

Copyright © 2021 DQMH® Consortium, LLC. All Rights Reserved.

Find more details on DQMH® Consortium website